

TP1 - Estructuras de Datos Avanzadas y Análisis Amortizado

Diseño de Algoritmos

Marisol

2025

Contents

1	Datos del trabajo	2
2	Ejercicio I-1: Montículo Binomial	2
2.1	A) Diseño UML de clases	2
2.2	B) Algoritmos	3
2.3	C) Análisis de eficiencia temporal	4
3	Ejercicio I-2: Conjuntos Disjuntos	4
3.1	A) Implementación con arreglos	4
3.2	B) Mejoras de eficiencia	5
3.3	C) Complejidad y justificación	6
4	Ejercicio I-3: Árbol TRIE - Diccionario de sinónimos	6
4.1	A) Diseño e implementación	6
4.2	B) Análisis de eficiencia	7
5	Ejercicio I-4: Análisis Amortizado	8
5.1	A) Contador binario extendido	8
5.2	B) Secuencia de operaciones con costos especiales	9
6	Conclusiones	9
7	Bibliografía y fuentes	10
8	Control de entrega	10

1 Datos del trabajo

- **Materia:** Diseño de Algoritmos
- **Trabajo práctico:** TP1
- **Tema:** Estructuras de Datos Avanzadas y Análisis Amortizado

2 Ejercicio I-1: Montículo Binomial

2.1 A) Diseño UML de clases

Dibujar el diagrama UML de clases para implementar un montículo binomial dinámico.

El diseño debe incluir las siguientes clases principales:

- `NodoBinomial`
- `ArbolBinomial`
- `HeapBinomial`

☐ Definir las clases principales.

☐ Identificar los atributos y métodos.

2.1.1 Diagrama UML

@startuml

```
class NodoBinomial {  
    ' Completar atributos y métodos  
}
```

```
class ArbolBinomial {  
    ' Completar atributos y métodos  
}
```

```
class HeapBinomial {
```

```

    ' Completar atributos y métodos
}

' Completar relaciones y multiplicidades

@enduml

```

2.1.2 Decisiones de diseño

- ¿Qué información almacena cada `NodoBinomial`?
- ¿Cómo se representa la raíz de un árbol binomial?
- ¿Cómo se enlazan los árboles binomiales dentro del heap?
- ¿Qué atributo permite localizar rápidamente el mínimo?

Escribir aquí las decisiones de diseño.

2.2 B) Algoritmos

Desarrollar los algoritmos para las siguientes operaciones:

- `insertar(x)`
- `unir(H1,H2)`
- `buscarMin()`
- `extraerMin()`
- `disminuirClave(x,k)`
- `eliminar(x)`

2.2.1 `insertar(x)`

Escribir aquí el pseudocódigo de `insertar(x)`.

2.2.2 `unir(H1,H2)`

Escribir aquí el pseudocódigo de `unir(H1,H2)`.

2.2.3 buscarMin()

Escribir aquí el pseudocódigo de buscarMin().

2.2.4 extraerMin()

Escribir aquí el pseudocódigo de extraerMin().

2.2.5 disminuirClave(x,k)

Escribir aquí el pseudocódigo de disminuirClave(x,k).

2.2.6 eliminar(x)

Escribir aquí el pseudocódigo de eliminar(x).

2.3 C) Análisis de eficiencia temporal

Analizar la eficiencia temporal de cada operación en notación O y justificarla.

Operación	Complejidad	Justificación
insertar(x)		
unir(H1,H2)		
buscarMin()		
extraerMin()		
disminuirClave(x,k)		
eliminar(x)		

Escribir aquí la justificación detallada de las complejidades.

3 Ejercicio I-2: Conjuntos Disjuntos

3.1 A) Implementación con arreglos

Implementar en pseudocódigo o Java las operaciones sobre conjuntos disjuntos usando una representación en arreglos:

- buscar(x) (find)
- fusionar(a,b) (union)

3.1.1 Representación utilizada

Explicar qué representa cada posición del arreglo y cómo se inicializan los conjuntos.

3.1.2 Implementación en pseudocódigo

Escribir aquí la implementación de `buscar(x)` y `fusionar(a,b)`.

3.1.3 Implementación en Java

```
public class ConjuntosDisjuntos {  
  
    // Completar atributos.  
  
    public ConjuntosDisjuntos(int n) {  
        // Completar inicialización.  
    }  
  
    public int buscar(int x) {  
        // Completar implementación.  
        return 0;  
    }  
  
    public void fusionar(int a, int b) {  
        // Completar implementación.  
    }  
}
```

3.2 B) Mejoras de eficiencia

Explicar cómo cambia la eficiencia al aplicar las siguientes técnicas:

3.2.1 Union by rank

Explicar cómo se decide qué raíz se convierte en hija de la otra y por qué esto evita que los árboles crezcan demasiado.

Escribir aquí la explicación de union by rank.

3.2.2 Path compression

Explicar cómo se modifican los enlaces durante la operación `buscar(x)` para acercar los nodos a la raíz.

Escribir aquí la explicación de path compression.

3.3 C) Complejidad y justificación

Calcular la complejidad de las operaciones en cada caso y justificar.

Variante	Complejidad de <code>buscar</code>	Complejidad de <code>fusionar</code>	Justificación
Representación básica			
Union by rank			
Path compression			
Union by rank + path compression			

Escribir aquí el análisis completo. Indicar, si corresponde, la complejidad amortizada.

4 Ejercicio I-3: Árbol TRIE - Diccionario de sinónimos

4.1 A) Diseño e implementación

Diseñar e implementar en Java un algoritmo que permita:

- ☐ Almacenar un diccionario de sinónimos en un TRIE.
- ☐ Agregar un sinónimo a una palabra existente.
- ☐ Mostrar todos los sinónimos de una palabra dada.
- ☐ Listar todas las palabras del diccionario sin mostrar sus sinónimos.

4.1.1 Estructura de datos

Describir qué información contiene cada nodo del TRIE y cómo se representa la lista de sinónimos asociada a una palabra.

Escribir aquí la descripción de la estructura.

4.1.2 Implementación en Java

```
import java.util.*;

public class TrieSinonimos {

    private static class Nodo {
        // Completar atributos del nodo.
```

```

    }

    private final Nodo raiz;

    public TrieSinonimos() {
        // Completar inicialización.
        raiz = null;
    }

    public void insertarPalabra(String palabra) {
        // Completar implementación.
    }

    public void agregarSinonimo(String palabra, String sinonimo) {
        // Completar implementación.
    }

    public List<String> mostrarSinonimos(String palabra) {
        // Completar implementación.
        return new ArrayList<>();
    }

    public List<String> listarPalabras() {
        // Completar implementación.
        return new ArrayList<>();
    }
}

```

4.1.3 Ejemplo de uso

```

// Completar con un ejemplo de creación del diccionario,
// inserción de palabras, carga de sinónimos y consultas.

```

4.2 B) Análisis de eficiencia

Analizar la eficiencia temporal de las operaciones **insertar**, **buscar** y **listar**.

Sea L la longitud de la palabra y sea N la cantidad de palabras almacenadas en el diccionario.

Operación	Complejidad	Justificación
Insertar una palabra		
Buscar una palabra		
Agregar un sinónimo		
Mostrar sinónimos		
Listar todas las palabras		

Escribir aquí la justificación de cada complejidad.

5 Ejercicio I-4: Análisis Amortizado

5.1 A) Contador binario extendido

Demostrar que, si se incluye la operación de decrementar una unidad en el contador binario visto en clase, una secuencia de n operaciones costará a lo sumo:

$$\Theta(n \cdot k)$$

donde k es la cantidad de dígitos del número binario.

5.1.1 Modelo del contador

Describir cómo se representa el contador y cómo funcionan las operaciones incrementar y decrementar.

Escribir aquí el modelo del contador.

5.1.2 Costo de incrementar

Analizar cuántos bits pueden cambiar durante un incremento.

5.1.3 Costo de decrementar

Analizar cuántos bits pueden cambiar durante un decremento.

5.1.4 Demostración para una secuencia de n operaciones

Escribir aquí la demostración y explicar por qué el costo total está acotado por $(n \cdot k)$

5.2 B) Secuencia de operaciones con costos especiales

5.2.1 Enunciado

Se realiza una secuencia de n operaciones sobre una estructura de datos dada. La operación i -ésima cuesta:

- $\text{costo}(i) = i$ si i es una potencia de 2.
- $\text{costo}(i) = 1$ en cualquier otro caso.

5.2.2 Método agregado

Usando el método agregado, calcular el costo amortizado de cada operación.

Escribir aquí el cálculo del costo total de la secuencia.

5.2.3 Costo total

La suma de los costos puede expresarse como:

$$T(n) = \sum_{i=1}^n \text{costo}(i)$$

Desarrollar y acotar la suma de los costos de las operaciones que son potencias de 2.

5.2.4 Costo amortizado por operación

El costo amortizado promedio de una operación es:

$$\hat{c} = \frac{T(n)}{n}$$

Completar el cálculo y expresar el resultado asintótico.

6 Conclusiones

Comparar las estructuras estudiadas y resumir qué técnica resulta más importante en cada caso.

7 Bibliografía y fuentes

- Material de la cátedra: TP1-1 - Análisis Amortizado - Diseño de Algoritmos, 2025.
- Agregar aquí los libros, apuntes, artículos o páginas consultadas.

8 Control de entrega

- ☐ Ejercicio I-1 completo.
- ☐ Diagrama UML incluido.
- ☐ Algoritmos del montículo binomial incluidos.
- ☐ Complejidades justificadas.
- ☐ Ejercicio I-2 completo.
- ☐ Union by rank explicado.
- ☐ Path compression explicado.
- ☐ Complejidades justificadas.
- ☐ Ejercicio I-3 completo.
- ☐ Código Java revisado y compilado.
- ☐ Complejidades del TRIE justificadas.
- ☐ Ejercicio I-4 completo.
- ☐ Demostraciones revisadas.
- ☐ Conclusiones escritas.
- ☐ Bibliografía agregada.
- ☐ Archivo exportado al formato solicitado por la cátedra.

9 Notas

Escribir aquí dudas, observaciones o temas para consultar.